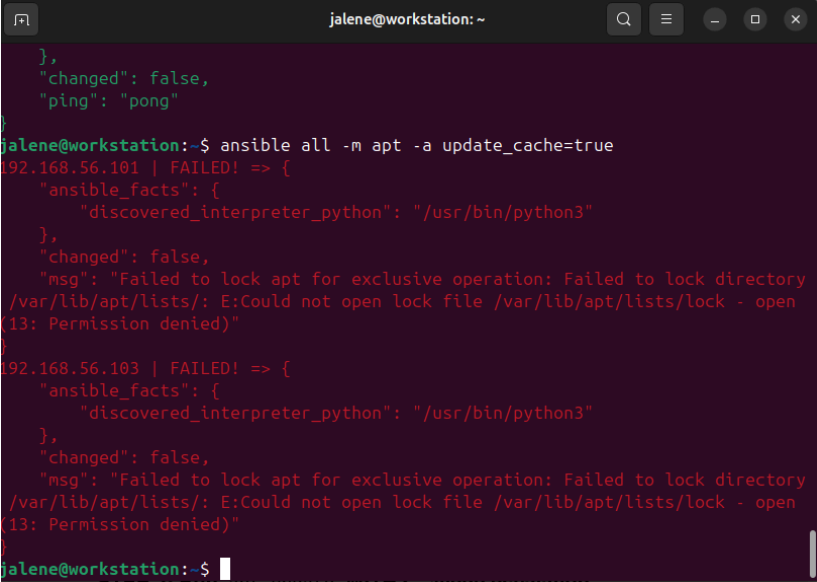


Name: Santiago, Jalene	Date Performed: January 8, 2026
Course/Section: CPE212 - CPE31S1	Date Submitted: January 8, 2026
Instructor: Engr. Robin Valenzuela	Semester and SY: 2nd Sem - 2025-2026
Activity 4: Running Elevated Ad hoc Commands	
1. Objectives: 1.1 Use commands that makes changes to remote machines 1.2 Use playbook in automating ansible commands	
2. Discussion: <i>Provide screenshots for each task.</i> Elevated Ad hoc commands So far, we have not performed ansible commands that make changes to the remote servers. We managed to gather facts and connect to the remote machines, but we still did not make changes on those machines. In this activity, we will learn to use commands that would install, update, and upgrade packages in the remote machines. We will also create a playbook that will be used for automations. Playbooks record and execute Ansible's configuration, deployment, and orchestration functions. They can describe a policy you want your remote systems to enforce, or a set of steps in a general IT process. If Ansible modules are the tools in your workshop, playbooks are your instruction manuals, and your inventory of hosts are your raw material. At a basic level, playbooks can be used to manage configurations of and deployments to remote machines. At a more advanced level, they can sequence multi-tier rollouts involving rolling updates, and can delegate actions to other hosts, interacting with monitoring servers and load balancers along the way. You can check this documentation if you want to learn more about playbooks. Working with playbooks — Ansible Documentation	
Task 1: Run elevated ad hoc commands 1. Locally, we use the command <i>sudo apt update</i> when we want to download package information from all configured resources. The sources often defined in <i>/etc/apt/sources.list</i> file and other files located in <i>/etc/apt/sources.list.d/</i> directory. So, when you run update command, it downloads the package information from the Internet. It is useful to get info on an updated version of packages or their dependencies. We can only run	

an apt update command in a remote machine. Issue the following command:

ansible all -m apt -a update_cache=true

What is the result of the command? Is it successful?

A terminal window titled 'jalene@workstation: ~' showing the output of an Ansible command. The command 'ansible all -m apt -a update_cache=true' was executed. The output shows that the command failed on two hosts (192.168.56.101 and 192.168.56.103) with the error message: 'Failed to lock apt for exclusive operation: Failed to lock directory /var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)'. The terminal also shows some JSON output for the first host, including 'changed': false and 'ping': 'pong'.

```
},
  "changed": false,
  "ping": "pong"
}
jalene@workstation:~$ ansible all -m apt -a update_cache=true
192.168.56.101 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "msg": "Failed to lock apt for exclusive operation: Failed to lock directory /var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)"
}
192.168.56.103 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "msg": "Failed to lock apt for exclusive operation: Failed to lock directory /var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)"
}
jalene@workstation:~$
```

Try editing the command and add something that would elevate the privilege. Issue the command *ansible all -m apt -a update_cache=true --become --ask-become-pass*. Enter the sudo password when prompted. You will notice now that the output of this command is a success. The *update_cache=true* is the same thing as running *sudo apt update*. The *--become* command elevate the privileges and the *--ask-become-pass* asks for the password. For now, even if we only have changed the packaged index, we were able to change something on the remote server.

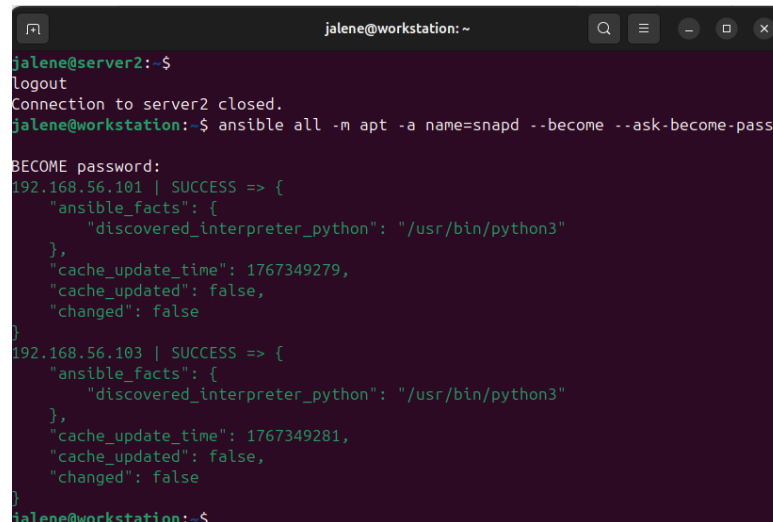
You may notice after the second command was executed, the status is CHANGED compared to the first command, which is FAILED.

2. Let's try to install VIM, which is an almost compatible version of the UNIX editor Vi. To do this, we will just changed the module part in 1.1 instruction. Here is the command: *ansible all -m apt -a name=vim-nox --become --ask-become-pass*. The command would take some time after typing the password because the local machine instructed the remote servers to actually install the package.

- 2.1 Verify that you have installed the package in the remote servers. Issue the command `which vim` and the command `apt search vim-nox` respectively. Was the command successful?
- 2.2 Check the logs in the servers using the following commands: `cd /var/log`. After this, issue the command `ls`, go to the folder `apt` and open `history.log`. Describe what you see in the `history.log`.
3. This time, we will install a package called `snapd`. Snap is pre-installed in Ubuntu system. However, our goal is to create a command that checks for the latest installation package.

3.1 Issue the command: `ansible all -m apt -a name=snapd --become --ask-become-pass`

Can you describe the result of this command? Is it a success? Did it change anything in the remote servers?



```
jalene@workstation: ~  
jalene@server2:~$  
logout  
Connection to server2 closed.  
jalene@workstation:~$ ansible all -m apt -a name=snapd --become --ask-become-pass  
BECOME password:  
192.168.56.101 | SUCCESS => {  
  "ansible_facts": {  
    "discovered_interpreter_python": "/usr/bin/python3"  
  },  
  "cache_update_time": 1767349279,  
  "cache_updated": false,  
  "changed": false  
}  
192.168.56.103 | SUCCESS => {  
  "ansible_facts": {  
    "discovered_interpreter_python": "/usr/bin/python3"  
  },  
  "cache_update_time": 1767349281,  
  "cache_updated": false,  
  "changed": false  
}  
jalene@workstation:~$
```

It was successful but the change state is false which means there were no changes when I ran the command.

- 3.2 Now, try to issue this command: `ansible all -m apt -a "name=snapd state=latest" --become --ask-become-pass`
Describe the output of this command. Notice how we added the command `state=latest` and placed them in double quotations.
4. At this point, make sure to commit all changes to GitHub.

TASK 1

```
jalene@workstation:~$ ansible all -m ping
192.168.56.101 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
192.168.56.103 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```

```
jalene@workstation:~$ ansible all -m apt -a update_cache=true --become --ask-become-pass
BECOME password:
192.168.56.101 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1767349279,
  "cache_updated": true,
  "changed": true
}
192.168.56.103 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1767349281,
  "cache_updated": true,
  "changed": true
}
jalene@workstation:~$
```

```

}
jalene@workstation:~$ ansible all -m apt -a name=vim-nox --become --ask-become-p
ass
BECOME password:
192.168.56.103 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1767349281,
  "cache_updated": false,
  "changed": true,
  "stderr": "",
  "stderr_lines": [],
  "stdout": "Reading package lists...\nBuilding dependency tree...\nReading st
ate information...\nThe following package was automatically installed and is no
longer required:\n  libllvm19\nUse 'sudo apt autoremove' to remove it.\nThe foll
owing additional packages will be installed:\n  fonts-lato javascript-common lib
js-jquery liblua5.1-0 libruby libruby3.2\n  libsodium23 rake ruby ruby-net-telne
t ruby-rubygems ruby-sdbm ruby-webrick\n  ruby-xmlrpc ruby3.2 rubygems-integrati
on vim-runtime\nSuggested packages:\n  apache2 | lighttpd | httpd ri ruby-dev bu
ndler cscope vim-doc\nThe following NEW packages will be installed:\n  fonts-lat
o javascript-common libjs-jquery liblua5.1-0 libruby libruby3.2\n  libsodium23 r
ake ruby ruby-net-telnet ruby-rubygems ruby-sdbm ruby-webrick\n  ruby-xmlrpc rub
y3.2 rubygems-integration vim-nox vim-runtime\n0 upgraded, 18 newly installed, 0

```

```

}
192.168.56.101 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1767349279,
  "cache_updated": false,
  "changed": true,
  "stderr": "",
  "stderr_lines": [],
  "stdout": "Reading package lists...\nBuilding dependency tree...\nReading st
ate information...\nThe following package was automatically installed and is no
longer required:\n  libllvm19\nUse 'sudo apt autoremove' to remove it.\nThe foll
owing additional packages will be installed:\n  fonts-lato javascript-common lib
js-jquery liblua5.1-0 libruby libruby3.2\n  libsodium23 rake ruby ruby-net-telne
t ruby-rubygems ruby-sdbm ruby-webrick\n  ruby-xmlrpc ruby3.2 rubygems-integrati
on vim-runtime\nSuggested packages:\n  apache2 | lighttpd | httpd ri ruby-dev bu
ndler cscope vim-doc\nThe following NEW packages will be installed:\n  fonts-lat
o javascript-common libjs-jquery liblua5.1-0 libruby libruby3.2\n  libsodium23 r
ake ruby ruby-net-telnet ruby-rubygems ruby-sdbm ruby-webrick\n  ruby-xmlrpc rub
y3.2 rubygems-integration vim-nox vim-runtime\n0 upgraded, 18 newly installed, 0
to remove and 7 not upgraded.\nNeed to get 18.6 MB of archives.\nAfter this ope
ration, 84.2 MB of additional disk space will be used.\nGet:1 http://ph.archive.
ubuntu.com/ubuntu noble/main amd64 fonts-lato all 3.015-1 [2781 kB]\nGet:2 http:

```

VERIFICATION

```
jalene@server1:~$ apt search vim-nox
Sorting... Done
Full Text Search... Done
vim-nox/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.9 amd64 [installed]
  Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.9 amd64 [installed,automatic]
  Vi IMproved - enhanced vi editor - compact version
```



```
jalene@server1: /var/log/apt
GNU nano 7.2 history.log
Start-Date: 2026-01-02 10:24:19
Commandline: /usr/bin/apt-get -y -o Dpkg::Options::=--force-confdef -o Dpkg::Option>
Requested-By: jalene (1000)
Install: ruby-sdbm:amd64 (1.0.0-5build4, automatic), liblua5.1-0:amd64 (5.1.5-9buil>
End-Date: 2026-01-02 10:24:32
```

```
jalene@server2:~$ which vim
/usr/bin/vim
jalene@server2:~$ apt search vim-nox
Sorting... Done
Full Text Search... Done
vim-nox/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.9 amd64 [installed]
  Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.9 amd64 [installed,aut
omatic]
  Vi IMproved - enhanced vi editor - compact version

jalene@server2:~$
```



```
jalene@server2: /var/log/apt
GNU nano 7.2 history.log
Start-Date: 2026-01-02 10:24:27
Commandline: /usr/bin/apt-get -y -o Dpkg::Options::=--force-confdef -o Dpkg::Opt>
Requested-By: jalene (1000)
Install: ruby-sdbm:amd64 (1.0.0-5build4, automatic), liblua5.1-0:amd64 (5.1.5-9b>
End-Date: 2026-01-02 10:24:40
```

INSTALLING

```
jalene@workstation: ~  
jalene@server2:~$  
logout  
Connection to server2 closed.  
jalene@workstation:~$ ansible all -m apt -a name=snapt --become --ask-become-pass  
BECOME password:  
192.168.56.101 | SUCCESS => {  
  "ansible_facts": {  
    "discovered_interpreter_python": "/usr/bin/python3"  
  },  
  "cache_update_time": 1767349279,  
  "cache_updated": false,  
  "changed": false  
}  
192.168.56.103 | SUCCESS => {  
  "ansible_facts": {  
    "discovered_interpreter_python": "/usr/bin/python3"  
  },  
  "cache_update_time": 1767349281,  
  "cache_updated": false,  
  "changed": false  
}  
jalene@workstation:~$  
jalene@workstation:~$ ansible all -m apt -a "name=snapt state=latest" --become --ask-become-pass  
BECOME password:  
192.168.56.101 | SUCCESS => {  
  "ansible_facts": {  
    "discovered_interpreter_python": "/usr/bin/python3"  
  },  
  "cache_update_time": 1767349279,  
  "cache_updated": false,  
  "changed": false  
}  
192.168.56.103 | SUCCESS => {  
  "ansible_facts": {  
    "discovered_interpreter_python": "/usr/bin/python3"  
  },  
  "cache_update_time": 1767349281,  
  "cache_updated": false,  
  "changed": false  
}  
jalene@workstation:~$
```

Task 2: Writing our First Playbook

1. With ad hoc commands, we can simplify the administration of remote servers. For example, we can install updates, packages, and applications, etc. However, the real strength of ansible comes from its playbooks. When we write a playbook, we can define the state that we want our servers to be in and the place or commands that ansible will carry out to bring to that state. You can use an editor to create a playbook. Before we proceed, make sure that you are in the directory of the repository that we use in the previous activities (*CPE232_yourname*). Issue the command *nano install_apache.yml*. This will create a playbook file called *install_apache.yml*. The .yml is the basic standard extension for playbook files.

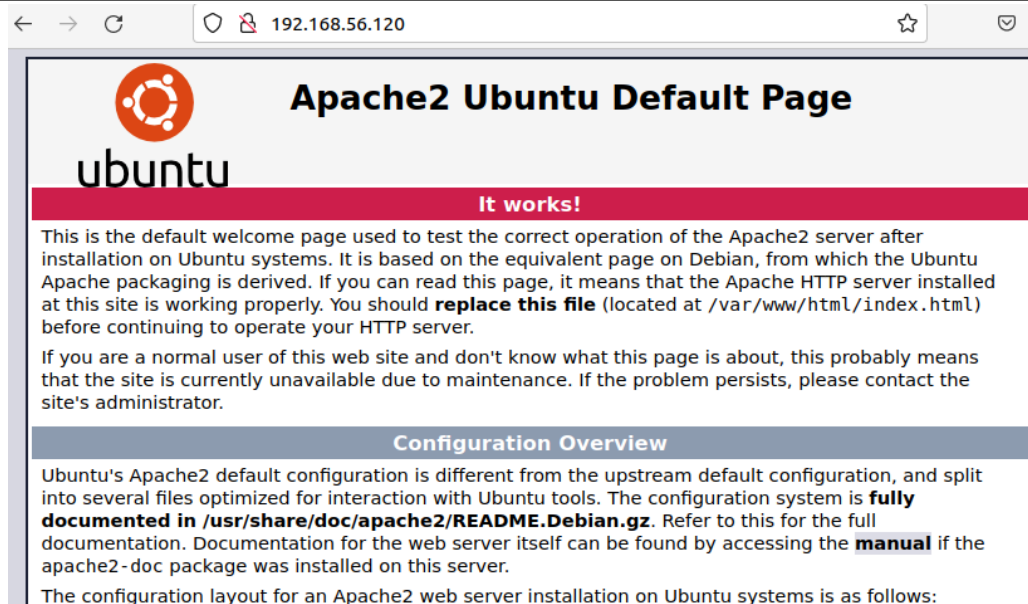
When the editor appears, type the following:

```
GNU nano 4.8          install_apache.yml
--
- hosts: all
  become: true
  tasks:

    - name: install apache2 package
      apt:
        name: apache2
```

Make sure to save the file. Take note also of the alignments of the texts.

2. Run the yml file using the command: *ansible-playbook --ask-become-pass install_apache.yml*. Describe the result of this command.
3. To verify that apache2 was installed automatically in the remote servers, go to the web browsers on each server and type its IP address. You should see something like this.



4. Try to edit the *install_apache.yml* and change the name of the package to any name that will not be recognized. What is the output? *The playbook fails because it does not recognize the misspelled package.*
5. This time, we are going to put additional tasks into our playbook. Edit the *install_apache.yml*. As you can see, we are now adding an additional command, which is the *update_cache*. This command updates existing package-indexes on a supporting distro but not upgrading installed-packages (utilities) that were being installed.

```
---
- hosts: all
  become: true
  tasks:

    - name: update repository index
      apt:
        update_cache: yes

    - name: install apache2 package
      apt:
        name: apache2
```

Save the changes to this file and exit.

6. Run the playbook and describe the output. Did the new command change anything on the remote servers? *The playbook successfully updated the package cache. No changes were made to apache since it was already installed.*

7. Edit again the *install_apache.yml*. This time, we are going to add PHP support for the apache package we installed earlier.

```
---
- hosts: all
  become: true
  tasks:

    - name: update repository index
      apt:
        update_cache: yes

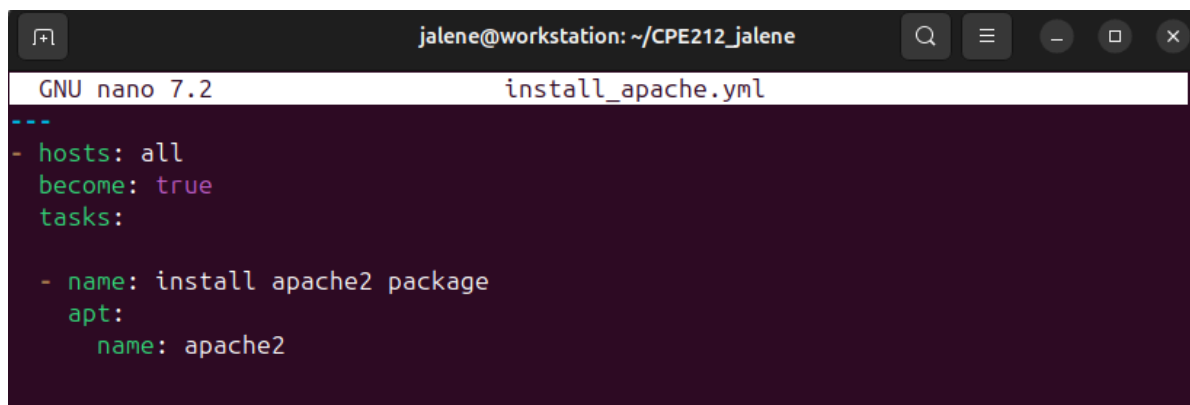
    - name: install apache2 package
      apt:
        name: apache2

    - name: add PHP support for apache
      apt:
        name: libapache2-mod-php
```

Save the changes to this file and exit.

8. Run the playbook and describe the output. Did the new command change anything on the remote servers? *The playbook successfully installed PHP support for Apache on the remote servers.*
9. Finally, make sure that we are in sync with GitHub. Provide the link to your GitHub repository.

TASK 2



```
jalene@workstation: ~/CPE212_jalene
GNU nano 7.2 install_apache.yml
---
- hosts: all
  become: true
  tasks:

    - name: install apache2 package
      apt:
        name: apache2
```

```

jalene@workstation:~$ cd CPE212_jalene
jalene@workstation:~/CPE212_jalene$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

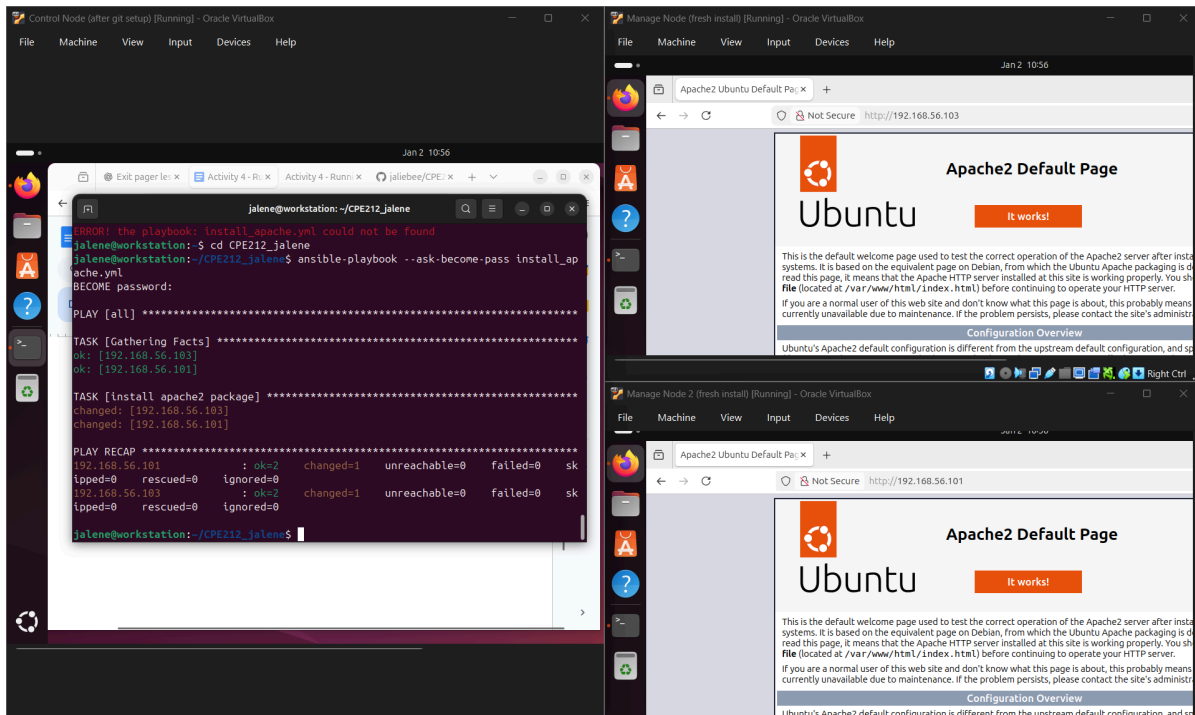
TASK [Gathering Facts] *****
ok: [192.168.56.103]
ok: [192.168.56.101]

TASK [install apache2 package] *****
changed: [192.168.56.103]
changed: [192.168.56.101]

PLAY RECAP *****
192.168.56.101      : ok=2    changed=1    unreachable=0    failed=0    sk
ipped=0    rescued=0    ignored=0
192.168.56.103      : ok=2    changed=1    unreachable=0    failed=0    sk
ipped=0    rescued=0    ignored=0

jalene@workstation:~/CPE212_jalene$

```



INSTALLING UNRECOGNIZED PACKAGE

```
jalene@workstation: ~/CPE212_jalene
GNU nano 7.2 install_apache.yml
---
- hosts: all
  become: true

  tasks:
    - name: install apache2 package
      apt:
        name: apache123
        state: present

TASK [install apache2 package] *****
fatal: [192.168.56.101]: FAILED! => {"changed": false, "msg": "No package matchin
g 'apache123' is available"}
fatal: [192.168.56.103]: FAILED! => {"changed": false, "msg": "No package matchin
g 'apache123' is available"}

PLAY RECAP *****
192.168.56.101      : ok=1    changed=0    unreachable=0    failed=1    sk
ipped=0    rescued=0    ignored=0
192.168.56.103      : ok=1    changed=0    unreachable=0    failed=1    sk
ipped=0    rescued=0    ignored=0
```

ADDING UPDATE CACHE

```
jalene@workstation: ~/CPE212_jalene
GNU nano 7.2 install_apache.yml
---
- name: Install Apache Web Server
  hosts: all
  become: yes

  tasks:
    - name: Update apt cache
      apt:
        update_cache: yes

    - name: Install apache2
      apt:
        name: apache2
        state: present
```

```

PLAY [Install Apache Web Server] *****

TASK [Gathering Facts] *****
ok: [192.168.56.103]
ok: [192.168.56.101]

TASK [Update apt cache] *****
changed: [192.168.56.103]
changed: [192.168.56.101]

TASK [Install apache2] *****
ok: [192.168.56.101]
ok: [192.168.56.103]

PLAY RECAP *****
192.168.56.101 : ok=3    changed=1    unreachable=0    failed=0    sk
ipped=0    rescued=0    ignored=0
192.168.56.103 : ok=3    changed=1    unreachable=0    failed=0    sk
ipped=0    rescued=0    ignored=0

jalene@workstation:~/CPE212_jalene$

```

ADDING PHP SUPPORT

```
jalene@workstation: ~/CPE212_jalene
GNU nano 7.2 install_apache.yml *
---
- name: Install Apache with PHP support
  hosts: all
  become: yes

  tasks:
    - name: Update apt cache
      apt:
        update_cache: yes

    - name: Install apache2
      apt:
        name: apache2
        state: present

    - name: Install PHP support
      apt:
        name: libapache2-mod-php
        state: present

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

```
jalene@workstation: ~/CPE212_jalene
jalene@workstation:~/CPE212_jalene$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:

PLAY [Install Apache with PHP support] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]
ok: [192.168.56.103]

TASK [Update apt cache] *****
changed: [192.168.56.103]
changed: [192.168.56.101]

TASK [Install apache2] *****
ok: [192.168.56.101]
ok: [192.168.56.103]

TASK [Install PHP support] *****
changed: [192.168.56.103]
changed: [192.168.56.101]

PLAY RECAP *****
192.168.56.101      : ok=4    changed=2    unreachable=0    failed=0    sk
ipped=0    rescued=0    ignored=0
192.168.56.103      : ok=4    changed=2    unreachable=0    failed=0    sk
ipped=0    rescued=0    ignored=0
```

GIT PUSH



jaliebee Task 2: Apache installation playbook

fb1d79b · 1 minute ago



README.md

new commit

3 weeks ago



install_apache.yml

Task 2: Apache installation ...

1 minute ago



readme2.md

for laptop

2 weeks ago

Reflections:

Answer the following:

1. What is the importance of using a playbook?

Using a playbook is important because it allows administrators to define tasks in a structured, readable, and repeatable format, ensuring that system configurations are applied consistently across multiple remote machines. Playbooks help reduce human error, save time, and make it easier to manage elevated commands by automating tasks that would otherwise be done manually on each system.

2. Summarize what we have done on this activity.

In this activity, we executed elevated ad hoc Ansible commands to update package repositories and install software such as VIM, snapd, and apache on remote machines using privilege escalation. We observed how commands behave with and without elevated privileges and verified changes through logs and system checks. We also created and modified an Ansible playbook to automate package installation, updates, and service configuration, demonstrating how playbooks simplify and standardize remote system management.